

Apuntes de Node y Express

Servidor

```
// -----  
// CREAR SERVIDOR CON NODE Y EXPRESS  
// -----  
  
// En la carpeta del proyecto, abrir terminal e ingresar:  
// npm init  
// Esto inicializa el paquete. Completar los datos que pide o saltar  
pasos con enter.  
// Los datos no ingresados adquieren valor por defecto. Para  
cambiarlos, editar archivo package.json  
  
// En terminal, ingresar:  
// npm install express  
// Para instalar Express y sus dependencias.  
  
// Importar Express:  
const express = require('express');  
const app = express();  
  
// Importar objeto para simular base de datos:  
const { productList } = require('./productList.js');  
  
// Habilitar escucha del servidor en un puerto:  
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => {  
  console.log(`El servidor esta escuchando en el puerto  
${PORT}...`);  
});  
  
// -----  
// ROUTING  
// -----  
  
// Ruta principal (raiz, home o index):  
app.get('/', (req, res) => {  
  res.send('Respuesta del servidor.');});
```

```

// Ruta del recurso sin modificaciones:
// app.get('/api/products', (req, res) => {
//   res.json(productList.products);
// });

// Para enviar respuestas en formato json:
// res.json();
// Es equivalente a:
// res.json();

// Ruta del recurso con parametro query:
app.get('/api/products', (req, res) => {

  // Si la ruta incluye el parametro query:
  // Ejemplo: ?sort=name
  if (req.query.sort == 'name') {
    let results = productList.products;
    results = results.sort(function (a, b) {
      let nameA = a.name.toLowerCase();
      let nameB = b.name.toLowerCase();
      return (nameA < nameB) ? -1 : (nameA > nameB) ? 1 : 0;
    });
    return res.json(results);
  }

  // Si la ruta no incluye el parametro query:
  res.json(productList.products);
});

// Ruta con parametro de url:
app.get('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let results = productList.products.filter(product => product.id ==
id);

  if (results.length == 0) {
    return res.status(404).send(`Product ID ${id} not found.`);
    // return res.status(404).end();
  }
  res.json(results);
}

```

```

});

// -----
// ROUTERS
// -----

// Para rutas largas que se repiten muchas veces:
const routerProducts = express.Router();
app.use('/api/products', routerProducts);
// Luego puedo reescribir:
// app.get('/api/products')
// Como:
// routerProducts.get('/')

// -----
// EJECUTAR SERVIDOR
// -----

// Para testear servidor, ingresar en terminal (cmd):
// nodemon server.js
// Y luego abrir en navegador:
// localhost:3000
// Esta es la ruta raiz, home o index.

```

CRUD

```

// -----
// CRUD (Create, Read, Update, Delete)
// -----

// Metodos:
// Get - Leer
// Post - Crear
// Put, Patch - Actualizar
// Delete - Eliminar

// Para manejar las solicitudes con json,
// incluir middleware antes del routing:
app.use(express.json());

// -----

```

```

// METODO GET
// -----

// Manejo de solicitud por metodo get:
app.get('/api/products', (req, res) => {
  res.json(productList.products);
});

// Solicitud: ingresar a la url indicada en la ruta.
// http://localhost:3000/api/products

// -----
// METODO POST
// -----

// Manejo de solicitud por metodo post:
app.post('/api/products', (req, res) => {
  let newProduct = req.body;
  productList.push(newProduct);
  res.json(productList.products);
});

// Solicitud:
// POST http://localhost:3000/api/products HTTP/1.1
// Content-Type: application/json
// {
//   "id": 50921,
//   "name": "Chevrolet Onix Joy",
//   "description": "Generación 2019, variedad de colores. Motor
1.0, ideal para ciudad.",
//   "cost": 13500,
//   "currency": "USD",
//   "soldCount": 14,
//   "image": "img/prod50921_1.jpg"
// }

// -----
// METODO PUT
// -----

// Manejo de solicitud por metodo put:

```

```

app.put('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let updatedProduct = req.body;
  let index = productList.products.findIndex(product => product.id
== id);

  if (index >= 0) {
    productList.products[index] = updatedProduct;
  }
  res.json(productList.products);
});

// Solicitud:
// PUT http://localhost:3000/api/products/50921 HTTP/1.1
// Content-Type: application/json
// {
//   "id": 50921,
//   "name": "Chevrolet Onix Joy",
//   "description": "Generación 2019, variedad de colores. Motor
1.0, ideal para ciudad.",
//   "cost": 13500,
//   "currency": "USD",
//   "soldCount": 14,
//   "image": "img/prod50921_1.jpg"
// }

// -----
// METODO PATCH
// -----

// Manejo de solicitud por metodo patch:
app.patch('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let updatedProperty = req.body;
  let index = productList.products.findIndex(product => product.id
== id);

  if (index >= 0) {
    let productToUpdate = productList.products[index];
    Object.assign(productToUpdate, updatedProperty);
  }
}

```

```

    res.json(productList.products);
  });

// Solicitud:
// PATCH http://localhost:3000/api/products/50921 HTTP/1.1
// Content-Type: application/json
// {
//   "soldCount": 14,
// }

// -----
// METODO DELETE
// -----

// Manejo de solicitud por metodo delete:
app.delete('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let index = productList.products.findIndex(product => product.id
== id);

  if (index >= 0) {
    productList.products.splice(index, 1);
  }
  res.json(productList.products);
});

// Solicitud:
// DELETE http://localhost:3000/api/products/50921 HTTP/1.1

```